claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a33fdf55c8e5da9e8.jsonl`
- SHA-256: `f499f9e9fa58c1bc3350c9d88a83d26e0ab156a27c5c4e3435eedfd7ba71ba14`
- Source modified: `2026-06-22T15:06:27+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T15:05:03.850Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: dispatch-correctness
Severity: minor
Title: Cloud-vs-local video_id divergence is silently tolerated, not validated or logged
File: backend/main.py:520-524
Detail: The API ingests under its LOCAL video_id (hashed from acquire_local_input bytes) but reads result.outputs_uri, which CloudRunCompute builds from the WORKER's video_id (backend/compute/cloud_run.py:129, hashed from the worker's bucket fetch). ComputeJobSpec's docstring asserts these are identical ('the MD5 the worker will (re)compute from the SAME input bytes'). In the normal case they match. But there is no assertion/log if result.video_id != video_id. If the bucket object is overwritten between the API fetch and the worker fetch (concurrent re-upload), the two diverge: segments still ingest correctly under the local id (the read targets the worker's outputs_uri), but report.json/the done-marker land under the worker's id ? orphaned from the DB video row under the local id, and any downstream consumer that trusts the marker/report video_id will mismatch. The existing tests (test_api_cloud_dispatch.py) actually exercise the divergent case (cloud id 'vidCloud' vs the b'FAKEVIDEOBYTES' local hash) and rely on this read-cloud/write-local split working ? so it is by-design, but the silent divergence deserves a defensive log/guard.
Fix: After run_infer, if result.video_id and result.video_id != video_id, log a WARNING (input bytes changed between API hash and worker hash) so the orphaned-report condition is diagnosable; optionally treat a large divergence as a failure.

2. assistant (2026-06-22T15:05:06.762Z)

I'll verify this finding by reading the cited code in the bhi-cloudrun repo.

3. user (2026-06-22T15:05:10.339Z)

---BRANCH---
feat/serving-api-cloud-dispatch
---HEAD---
304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF

4. user (2026-06-22T15:05:10.537Z)

```
 backend/main.py                  | 100 ++++++++++++++++++
 tests/test_api_cloud_dispatch.py | 212 ++++++++++++++++++++++++++++++++++++++++
 2 files changed, 312 insertions(+)
```

5. assistant (2026-06-22T15:05:12.441Z)

Let me read the cited code in the diff.

6. user (2026-06-22T15:05:13.834Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
 import os
 import sys
 import argparse
+import csv
 import json
 import logging
 import shutil
 import socket
+import tempfile
 import threading
 import uuid
 import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
     return FileResponse(path, media_type=media, filename=name)


+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
(cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                    "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                           play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
+    == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
```

```
intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
+
+    compute = get_compute_backend(global_config)
+    if compute is None:
+        return None  # default-OFF ? run the in-process segmenter below (unchanged)
+
+    storage_cfg = getattr(global_config, "storage", None)
+
+    def _failed(msg: str) -> Dict[str, Any]:
+        # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+        return {"video_id": video_id, "status": "failed", "message": msg,
+                "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+    # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+    try:
+        input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+    except ValueError as e:
+        return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+    if input_ref.backend == "local":
+        return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+                       "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+    if not out_root:
+        return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+                       "? that's where the Cloud Run job writes the result.")
+
+    notify("dispatching (cloud_run)")
+    logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+    spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+    try:
+        result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
+    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+        return _failed(f"cloud dispatch error: {e}")
+    if result.returncode != 0:
+        return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+    notify("ingesting (cloud_run)")
+    storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+    _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+    segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+    _finalize_reingest(video_id, segments, play_wm, cand_wm)
+    return {"video_id": video_id, "status": "success",
+            "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
```

```
+            "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
         # if the run yields segments; otherwise drop the new partial rows and keep the old.
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+        # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+        # they're dropped on the cloud path in this slice ? a documented follow-up.)
+        remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+        if remote is not None:
+            segmentation_ran = True
+            response = remote
+            return response
         segmenter = segmenter_cls(db_instance, global_config)
         try:
             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
```

7. assistant (2026-06-22T15:05:16.072Z)

Now let me look at the cited lines 520-524 and understand the full `_execute_processing` flow, especially how `video_id` is computed and where the read happens.

8. user (2026-06-22T15:05:16.947Z)

```
500             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
501                            "can't be read by the Cloud Run job. Put the video in your bucket
first.")
502         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
503         if not out_root:
504             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
505                            "? that's where the Cloud Run job writes the result.")
506
507         notify("dispatching (cloud_run)")
508         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
509         spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
510         try:
511             result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
512         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
513             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
514             return _failed(f"cloud dispatch error: {e}")
515         if result.returncode != 0:
516             return _failed(f"cloud job failed (returncode {result.returncode}).")
517
518         notify("ingesting (cloud_run)")
519         storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
```

```
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
570                     "video_id": video_id,
571                     "status": "failed",
572                     "message": "Video failed validation checks.",
573                     "results": [r.model_dump() for r in val_results],
```

```
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
            if remote is not None:
                segmentation_ran = True
                response = remote
                return response
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
```

```
627                     "status": "failed",
628                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
629                     "results": [r.model_dump() for r in val_results],
```

9. assistant (2026-06-22T15:05:19.280Z)

Now let me look at the cited cloud_run.py to confirm how the worker's video_id is computed and where outputs_uri comes from, and the ComputeJobSpec docstring.

10. user (2026-06-22T15:05:20.330Z)

```
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                                "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                              project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                      self._job_name, execution, done_uri)
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128                 rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
```

```
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156
```

11. assistant (2026-06-22T15:05:23.588Z)

The finding's claim about cloud_run.py:129 is confirmed: `outputs_uri = f"{out_root}/outputs/{video_id}/"` where `video_id` comes from the worker's marker. Now let me check the ComputeJobSpec docstring to verify the "identical MD5" claim.

12. user (2026-06-22T15:05:25.537Z)

```
backend/compute/base.py:21:class ComputeJobSpec:
backend/compute/base.py:22:    """One inference job, profile-agnostic. ``video_id`` is the MD5
the worker will (re)compute
backend/compute/base.py:23:    from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
backend/compute/base.py:34:class ComputeResult:
Binary file backend/compute/__pycache__/base.cpython-313.pyc matches
```

13. user (2026-06-22T15:05:29.115Z)

```
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str                          # input (gs://? for cloud; a path/local://
for local)
27          out_uri: str                            # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
```

```
39
40          returncode: int
41          outputs_uri: str = ""
42          video_id: str = ""
43          report: dict = field(default_factory=dict)
44
45
46      class ComputeBackend(ABC):
47          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49          execution identically."""
50
51          @abstractmethod
52          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53              """Run ``spec`` to completion and return its ``ComputeResult``."""
54
```

14. assistant (2026-06-22T15:05:34.972Z)

The docstring claim is confirmed. Now let me verify the critical claim in the finding: that the API ingests under the LOCAL video_id but reads from `result.outputs_uri` (built from the worker's video_id). Let me re-confirm:

- Line 549: `video_id = compute_video_id(local_video)` ? LOCAL hash from `acquire_local_input`.
- Line 520: `_ingest_remote_segments(video_id, result.outputs_uri, ...)` ? ingests under LOCAL `video_id`, reads from `result.outputs_uri` (worker's id).
- `result.video_id` is available (the worker's id) but is NOT compared to the local `video_id`.

Now let me confirm the test actually exercises the divergent case as claimed.

15. user (2026-06-22T15:05:36.317Z)

```
42:    def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44:        self.video_id, self.returncode = video_id, returncode
53:            outputs = os.path.join(out_uri, "outputs", self.video_id)
56:                json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
65:            json.dump({"video_id": self.video_id, "returncode": self.returncode,
86:    fake_local.write_bytes(b"FAKEVIDEOBYTES")
114:    rows = db.query_play_segments(out["video_id"], {})
150:    assert db.query_play_segments(out["video_id"], {}) == []
195:    fake = _FakeCloudClient(video_id="vidCloud2")
205:    video_id = job["video_id"]
206:    assert video_id
208:    q = client.post("/api/query", json={"video_id": video_id})
```

16. assistant (2026-06-22T15:05:39.997Z)

Let me read the test's `_FakeCloudClient` and key assertions to understand how the divergent case is exercised and what `outputs_uri`/`video_id` the API returns.

17. user (2026-06-22T15:05:40.702Z)

```
40          into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
41
42          def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
43              self.calls: list[list[str]] = []
44              self.video_id, self.returncode = video_id, returncode
45              self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
46                                                         (8.74, 11.38, 3, "unforced_error")]
47
48          def run_job(self, job_name, argv, *, region, project=None):
```

```
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
                               "status": "success"}, f)
                with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "returncode": self.returncode,
                           "segment_count": len(self.rows), "status": "success"}, f)
            return "exec-1"


    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
        db = Database(str(tmp_path / "t.db"))
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate({
            "deployment": {"compute_target": "cloud_run"},
            "storage": {"backend": "local", "output_root": str(bucket)},
        })
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
str(fake_local))
        monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token."""
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(compute_pkg, "get_compute_backend",
                            lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


```

```
    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))

        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        rows = db.query_play_segments(out["video_id"], {})
        assert len(rows) == 2
        metas = [_meta(r) for r in rows]
        assert all(md["source"] == "cloud_run" for md in metas)
        assert {md.get("shot_count") for md in metas} == {5, 3}
        assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


    def test_cloud_dispatch_builds_correct_spec(cloud_env):
        _db, bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))

        assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
        assert argv[argv.index("--out-uri") + 1] == str(bucket)
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        joined = " ".join(argv)
        assert "deployment.compute_target=local_gpu" in joined  # container runs INLINE
(never re-dispatch)
        assert "indexing.detector_impl=native" in joined


    def test_cloud_failure_marks_failed_no_rows(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
        db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
        fake = _FakeCloudClient(returncode=1)
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
        assert out["status"] == "failed"
        assert "returncode 1" in out["message"]
        assert out["play_segments"] == []
        assert db.query_play_segments(out["video_id"], {}) == []


    def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "local_only.mp4")

        out = m._execute_processing(m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid"))
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == []  # never dispatched a local-only path

```

```
164
165     def test_default_off_runs_in_process(tmp_path, monkeypatch):
166         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
167         byte-identically (the gate is inert). No Cloud Run dispatch."""
168         db = Database(str(tmp_path / "t.db"))
169         cfg = AppConfig()  # compute_target defaults to ""
```

18. assistant (2026-06-22T15:05:47.220Z)

This confirms the divergence is real and by design. In the test:
- The local hash is `compute_video_id` of `b"FAKEVIDEOBYTES"` (some MD5).
- The worker/cloud `video_id` is `"vidCloud"`.
- `result.outputs_uri` is built as `<bucket>/outputs/vidCloud/` (worker id).
- The segments are ingested under the LOCAL `video_id` (line 520), and `out["video_id"]` is the LOCAL id (line 524).
- Line 114: `db.query_play_segments(out["video_id"], {})` succeeds ? the rows live under the LOCAL id, read from the worker's outputs_uri.

So the read-cloud/write-local split is confirmed real and by design. The finding's claim that `result.video_id != video_id` is never asserted/logged is also confirmed ? there's no comparison anywhere in `_maybe_dispatch_remote` or `_ingest_remote_segments`. Let me verify there's truly no such guard.

19. user (2026-06-22T15:05:48.640Z)

```
285:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
293:        db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
295:        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
490:        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
521:    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops
the old ones.
523:    _finalize_reingest(video_id, segments, play_wm, cand_wm)
618:            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
624:            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
636:        _finalize_reingest(video_id, segments, play_wm, cand_wm)
1162:            _finalize_reingest(video_id, segments, play_wm, cand_wm)
```

20. assistant (2026-06-22T15:05:58.235Z)

Confirmed: `result.video_id` is never referenced in main.py. The only consumer of the worker's id is implicitly via `result.outputs_uri`. The local `video_id` (line 549) is what everything is keyed under.

Now let me assess the severity and reality of the actual *defect*. The finding's core claim is sound on the facts, but I need to judge whether the divergence scenario is a real defect worth fixing or a theoretical nit. Let me think about the actual scenario:

1. In the **normal case**, the API fetches `gs://b/clip.mp4` to a local file, hashes it ? local `video_id`. The worker fetches the *same* `gs://b/clip.mp4` and hashes it ? worker `video_id`. Same bytes ? same MD5. They match. No problem.

2. The **divergence** requires the bucket object to be overwritten *between* the API's fetch and the worker's fetch (a concurrent re-upload of `gs://b/clip.mp4` with different bytes). Then:
   - Segments correctly ingest under the LOCAL id (read targets the worker's `outputs_uri`, which is correct for the read).
   - But `report.json` and the done-marker land under the worker's id in the bucket.
   - The DB video row + segment rows are under the LOCAL id ? orphaned from the bucket artifacts under the worker id.

This is a genuine, if narrow, consistency hazard. The finding correctly notes it's by-design (the test relies on the split), and the fix proposed is purely defensive: log a WARNING when `result.video_id != video_id`. That's a low-cost diagnosability improvement. Let me check whether `_finalize_reingest`/report consumers actually trust the marker id downstream, to gauge

severity.

21. user (2026-06-22T15:05:58.931Z)

```
285     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
286         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
287
288         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
289         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
290         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
291         """
292         if segments:
293             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
294         else:
```

22. assistant (2026-06-22T15:06:10.977Z)

I now have enough to render a verdict. The finding's factual claims all hold up:

1. **Local vs worker video_id divergence is real and by-design** ? confirmed at main.py:549 (local hash) vs cloud_run.py:121,129 (worker hash ? outputs_uri), and the test (`vidCloud` vs `FAKEVIDEOBYTES` hash) deliberately exercises the split.
2. **No assertion/log on `result.video_id != video_id`** ? confirmed; `result.video_id` is never referenced in main.py.
3. **The ComputeJobSpec docstring asserts they're identical** ? confirmed at base.py:22-24.
4. **The divergence trigger (bucket object overwritten between API fetch and worker fetch)** is plausible and would orphan report.json/done-marker (under worker id) from the DB rows (under local id).

The proposed fix (a defensive WARNING log when they diverge) is reasonable and low-cost, improving diagnosability of an otherwise-silent inconsistency.

Severity assessment: This is correctly rated **minor** (arguably even nit-leaning). The scenario requires a TOCTOU concurrent re-upload of the same bucket key during a single job, the feature is DEFAULT-OFF / M6 early-slice, segments still ingest *correctly* under the local id (no data corruption or wrong-rally), and the only consequence is orphaned bucket artifacts + a downstream-consumer mismatch that doesn't exist yet. No correctness bug in the happy path. But it is a real, by-design silent-divergence hazard that a one-line defensive log would make diagnosable, and the code already keys two different ids without ever reconciling them. I'll keep it as **minor** (the finding's stated severity), since it's a legitimate observability/robustness gap rather than a pure style nit.

23. user (2026-06-22T15:06:22.098Z)

Structured output provided successfully

24. assistant (2026-06-22T15:06:27.053Z)

Verdict submitted: the finding is real (is_real=true) at minor severity. All cited facts hold up against the code ? the local vs worker video_id divergence is genuinely silently tolerated with no assertion/log, it's by-design (the tests rely on the read-cloud/write-local split), and the proposed defensive WARNING is a reasonable low-cost diagnosability fix for a narrow TOCTOU consistency hazard in a default-OFF early slice.